LlamaIndex

Complete Guide — Concepts, Comparisons & Q&A

Core Concepts • Index Types • Retrievers • LangChain Comparison • Interview Q&A

2026

Section 1 — What is LlamaIndex?

LlamaIndex (formerly GPT Index) is an open-source Python framework designed specifically for connecting Large Language Models (LLMs) to your own data. While general-purpose LLM frameworks like LangChain can do many things, LlamaIndex is laser-focused on one problem:

> How do you efficiently load, index, and query your own documents using an LLM?

The Problem LlamaIndex Solves

LLMs like GPT-4 are trained on public internet data up to a cutoff date. They know nothing about your company's internal documents, your private codebase, or any private data. LlamaIndex solves this by creating a bridge:

```
Your private documents
        ↓
   LlamaIndex loads, chunks, embeds, and indexes them
        ↓
   User asks a question
        ↓
   LlamaIndex retrieves relevant chunks
        ↓
   LLM generates answer using those chunks
        ↓
   Answer grounded in YOUR data
```

LlamaIndex vs LangChain — The Key Philosophy Difference

This is the most important distinction to understand:

LangChain	LlamaIndex
General-purpose LLM toolkit	**Specialist data + LLM framework**
Builds chains of components	Thinks in terms of indexes
You wire everything together manually	Abstracts complexity behind the index
Best for: agents, tools, multi-step workflows	Best for: RAG, document Q&A, search
More verbose but more control	Less code, faster prototyping

Section 2 — Core Concepts

LlamaIndex is built around five fundamental building blocks. Understanding these is essential before writing any code.

1. Documents

A Document is the raw data loaded from any source — a PDF file, a web page, a database record, a Word document, an API response. Everything in LlamaIndex starts with Documents. They are the raw unprocessed input.

```
from llama_index.core import SimpleDirectoryReader

# Load all files from a folder:
documents = SimpleDirectoryReader('./docs').load_data()

# Load a single file:
documents = SimpleDirectoryReader(input_files=['report.pdf']).load_data()

# Each document has:
# document.text       → the raw text content
# document.metadata   → file name, page number, source, etc.
```

2. Nodes

Nodes are chunks of a Document after splitting. LlamaIndex automatically splits your documents into smaller pieces because LLMs have limited context windows and retrieving smaller, focused chunks gives better answers than retrieving entire documents.

In LangChain these are called 'Documents after splitting.' In LlamaIndex they are called 'Nodes.' Same concept, different name. Each Node carries a reference back to its source Document so citations are always possible.

```
from llama_index.core.node_parser import SentenceSplitter

splitter = SentenceSplitter(chunk_size=512, chunk_overlap=50)
nodes = splitter.get_nodes_from_documents(documents)

# Each node has:
# node.text            → the chunk text
# node.metadata        → inherited from parent document
# node.node_id         → unique identifier
# node.relationships   → links to previous/next nodes
```

3. Index

An Index is a structured representation of your Nodes that makes them efficiently queryable. When you build an index, LlamaIndex embeds each Node and organises them for fast retrieval. The index is the heart of LlamaIndex — it is what separates it from just storing text in a list.

The most commonly used index is VectorStoreIndex, which stores embeddings in a vector database and uses semantic similarity to find relevant nodes.

```
from llama_index.core import VectorStoreIndex

# Build index from documents (auto-splits into nodes + embeds):
index = VectorStoreIndex.from_documents(documents)

# Build index from pre-created nodes:
index = VectorStoreIndex(nodes)
```

4. Query Engine

A Query Engine sits on top of an Index and answers natural language questions. You ask a question, the query engine retrieves the most relevant Nodes from the Index, passes them to the LLM as context, and returns a grounded answer.

```
query_engine = index.as_query_engine()

response = query_engine.query('What are the main risks mentioned?')

print(response)               # the generated answer
print(response.source_nodes)  # exactly which chunks were used
```

5. Retriever

The Retriever is the component responsible for fetching relevant Nodes from the Index given a query. You can use it independently from the Query Engine when you want more control — for example, to inspect what was retrieved before generating an answer, or to combine multiple retrievers.

```
retriever = index.as_retriever(similarity_top_k=5)

# Get relevant nodes without generating an answer:
nodes = retriever.retrieve('What are the main risks?')

for node in nodes:
    print(node.score)       # similarity score
    print(node.text[:200])  # chunk content
```

> The full pipeline: Documents → split into Nodes → embedded into Index → Retriever fetches relevant Nodes → Query Engine generates answer using LLM

Section 3 — Index Types

One of LlamaIndex's biggest advantages over LangChain is its variety of built-in index types. Each is optimised for a different kind of question. Choosing the right index has a major impact on answer quality.

VectorStoreIndex — Semantic Similarity Search

The default and most commonly used index. Each Node is embedded into a vector and stored. At query time, the question is also embedded and the most semantically similar Nodes are retrieved.

Best for: questions that require understanding meaning, not just keyword matching. 'What were the key risks discussed?' rather than 'Find the word risk.'

```
from llama_index.core import VectorStoreIndex

index = VectorStoreIndex.from_documents(documents)
query_engine = index.as_query_engine(similarity_top_k=3)
response = query_engine.query('What were the main findings?')
```

SummaryIndex — Full Document Summarisation

Stores all Nodes in a sequential list and iterates through all of them when answering. Instead of retrieving a few relevant chunks, it reads everything. This is slower and more expensive but gives comprehensive answers for summarisation tasks.

Best for: 'Summarise this entire document', 'What is the overall theme?', questions that require reading the whole document rather than finding a specific answer.

```
from llama_index.core import SummaryIndex

index = SummaryIndex.from_documents(documents)
query_engine = index.as_query_engine(
    response_mode='tree_summarize'  # builds summary bottom-up
)
response = query_engine.query('Summarise this document')
```

KeywordTableIndex — Exact Keyword Matching

Extracts keywords from each Node and builds a keyword lookup table. At query time, keywords are extracted from the question and matched against the table. Fast and deterministic — always returns the same result for the same query.

Best for: technical documentation, legal documents, any domain where exact terminology matters more than semantic meaning. 'Find all mentions of GDPR Article 17.'

```
from llama_index.core import KeywordTableIndex

index = KeywordTableIndex.from_documents(documents)
query_engine = index.as_query_engine()
response = query_engine.query('What does the document say about GDPR Article 17?')
```

KnowledgeGraphIndex — Relationship-Based Retrieval

Extracts entities and relationships from documents and stores them as a knowledge graph. Useful when your data has complex interconnections — for example, a medical knowledge base where conditions, symptoms, and treatments are all related to each other.

Best for: complex interconnected data, multi-hop questions ('What diseases are caused by the same bacteria as X?'), knowledge bases where relationships between entities matter.

```
from llama_index.core import KnowledgeGraphIndex

index = KnowledgeGraphIndex.from_documents(
    documents,
    max_triplets_per_chunk=10
)
query_engine = index.as_query_engine(
    include_text=True,
    response_mode='tree_summarize'
)
```

Index Type	Best Use Case	Key Trade-off
VectorStoreIndex	Semantic questions over large docs	Needs embedding model
SummaryIndex	Full document summarisation	Slow — reads everything
KeywordTableIndex	Exact term lookup, technical docs	Misses synonyms
KnowledgeGraphIndex	Interconnected data, multi-hop Q&A	Complex to build

Section 4 — Key Features

Global Settings — Configure Once, Use Everywhere

One of LlamaIndex's most developer-friendly features. You set the LLM, embedding model, chunk size, and other parameters once globally. Every component — indexes, query engines, retrievers — automatically uses these settings. No need to pass the LLM to every single component like in LangChain.

```
from llama_index.core import Settings
from llama_index.llms.openai import OpenAI
from llama_index.embeddings.openai import OpenAIEmbedding

# Set once at the top of your app:
Settings.llm = OpenAI(model='gpt-4', temperature=0)
Settings.embed_model = OpenAIEmbedding(model='text-embedding-3-small')
Settings.chunk_size = 512
Settings.chunk_overlap = 50

# Now ALL components automatically use GPT-4 and the embedding model:
index = VectorStoreIndex.from_documents(documents)  # no need to pass llm
query_engine = index.as_query_engine()              # no need to pass llm

# To swap to a different model — change ONE line:
Settings.llm = OpenAI(model='gpt-4o')  # entire pipeline updated
```

Response Synthesizers — Control How Answers Are Generated

After retrieval, LlamaIndex gives you multiple strategies for synthesising the final answer from the retrieved nodes. This is something LangChain does not expose as cleanly.

Mode	How it works	Best for
compact	Fits all nodes into one LLM call	Speed and cost — default
refine	Generates answer from first node, refines using each next node	Accuracy on detailed questions
tree_summarize	Builds a bottom-up tree of summaries	Very long documents
simple_summarize	Truncates nodes to fit one LLM call	Quick rough answers
no_text	Returns retrieved nodes only, no LLM call	When you want raw retrieval

```
from llama_index.core.response_synthesizers import ResponseMode

# Most accurate:
query_engine = index.as_query_engine(response_mode=ResponseMode.REFINE)

# Fastest and cheapest:
query_engine = index.as_query_engine(response_mode=ResponseMode.COMPACT)

# Best for summarisation:
query_engine = index.as_query_engine(response_mode=ResponseMode.TREE_SUMMARIZE)
```

Source Nodes — Built-in Citation Tracking

Every response from a LlamaIndex query engine includes source_nodes — the exact chunks that were used to generate the answer. This is critical for enterprise AI applications where users need to verify answers against source material. LangChain requires custom implementation for this; LlamaIndex provides it out of the box.

```
response = query_engine.query('What are the key risks?')

# The generated answer:
print(str(response))

# Exactly which chunks were used:
for node in response.source_nodes:
    print(f'Score: {node.score:.3f}')          # relevance score
    print(f'Source: {node.metadata["file_name"]}')  # source document
    print(f'Text: {node.text[:300]}')           # the chunk content
    print('---')
```

Persistent Storage — Build Once, Load Many Times

By default, LlamaIndex builds the index in memory and it is lost on restart. For production applications, you persist the index to disk and load it on subsequent runs — avoiding the expensive embedding process every time the app starts.

```
from llama_index.core import (
    VectorStoreIndex, SimpleDirectoryReader,
    StorageContext, load_index_from_storage
)
import os

PERSIST_DIR = './index_storage'

if not os.path.exists(PERSIST_DIR):
    # First run — build index and save to disk
    documents = SimpleDirectoryReader('./docs').load_data()
    index = VectorStoreIndex.from_documents(documents)
    index.storage_context.persist(persist_dir=PERSIST_DIR)
    print('Index built and saved.')
else:
    # Subsequent runs — load from disk (fast)
    storage_context = StorageContext.from_defaults(persist_dir=PERSIST_DIR)
    index = load_index_from_storage(storage_context)
    print('Index loaded from disk.')

query_engine = index.as_query_engine()
```

Using External Vector Stores — Pinecone, ChromaDB, FAISS

For production RAG applications you connect LlamaIndex to an external vector database instead of in-memory storage. This enables scalability, persistence, and sharing the index across multiple services.

```
# With ChromaDB:
import chromadb
```

```
from llama_index.vector_stores.chroma import ChromaVectorStore
from llama_index.core import StorageContext

chroma_client = chromadb.PersistentClient(path='./chroma_db')
collection = chroma_client.get_or_create_collection('my_docs')
vector_store = ChromaVectorStore(chroma_collection=collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)

index = VectorStoreIndex.from_documents(
    documents,
    storage_context=storage_context
)
```

Section 5 — LangChain vs LlamaIndex: Full Comparison

Side-by-Side Code Comparison — RAG Pipeline

The same RAG pipeline built in both frameworks. Notice how LlamaIndex abstracts away more steps with sensible defaults, while LangChain gives explicit control over each component.

LangChain RAG Pipeline

```
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain.chains import RetrievalQA

# Step 1 — Load
loader = PyPDFLoader('document.pdf')
docs = loader.load()

# Step 2 — Split
splitter = RecursiveCharacterTextSplitter(chunk_size=512, chunk_overlap=50)
chunks = splitter.split_documents(docs)

# Step 3 — Embed and store
embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(chunks, embeddings)

# Step 4 — Build chain
llm = ChatOpenAI(model='gpt-4', temperature=0)
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vectorstore.as_retriever(search_kwargs={'k': 3})
)

# Step 5 — Query
answer = qa_chain.run('What are the main findings?')
```

LlamaIndex RAG Pipeline

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from llama_index.llms.openai import OpenAI
from llama_index.embeddings.openai import OpenAIEmbedding

# Step 1 — Configure once globally
Settings.llm = OpenAI(model='gpt-4', temperature=0)
Settings.embed_model = OpenAIEmbedding()
Settings.chunk_size = 512

# Step 2 — Load and index (splitting + embedding happen automatically)
documents = SimpleDirectoryReader(input_files=['document.pdf']).load_data()
index = VectorStoreIndex.from_documents(documents)

# Step 3 — Query
query_engine = index.as_query_engine(similarity_top_k=3)
```

```
response = query_engine.query('What are the main findings?')
print(str(response))
```

> LlamaIndex achieves the same result in roughly half the code. LangChain makes every step explicit; LlamaIndex handles the common steps automatically.

Full Feature Comparison Table

Feature	LangChain	LlamaIndex
Primary focus	General LLM framework	Data + LLM connection
RAG complexity	Manual pipeline setup	Abstracted, fast to build
Index types	Mainly vector store	4+ built-in index types
LLM configuration	Pass to each component	Global Settings object
Citation/sources	Custom implementation	Built-in source_nodes
Response strategies	Limited built-in	4 response modes
Agent support	Excellent — primary feature	Good but not primary
Tool integration	Excellent — 100s of tools	Good — growing ecosystem
Multi-step chains	Core strength	Not primary focus
Code verbosity	More verbose	Less code needed
Control/flexibility	More granular control	More abstraction
Best for	Agents, tools, workflows	RAG, document Q&A
Can they combine?	Yes — LlamaIndex as a LangChain tool	Yes — works both ways

When to Use Which — Decision Guide

Choose LangChain when...	Choose LlamaIndex when...
Building agents that use multiple tools	Primary goal is RAG over documents
Complex multi-step workflows	Fast prototyping with minimal code
Integrating with external APIs and services	Need built-in citation and source tracking
Need fine-grained control over every step	Summarising entire documents
Conversational agents with complex memory	Multiple index types for different queries
Combining RAG with other chains and tools	Complex document hierarchies and metadata

Section 6 — Complete Minimal RAG App with LlamaIndex

A production-ready minimal RAG application using LlamaIndex with persistent storage, ChromaDB as the vector store, and source citation. This is the pattern used in real AI Engineering projects.

```
# requirements.txt:
# llama-index
# llama-index-vector-stores-chroma
# llama-index-llms-openai
# llama-index-embeddings-openai
# chromadb

import os
import chromadb
from llama_index.core import (
    VectorStoreIndex, SimpleDirectoryReader,
    StorageContext, Settings
)
from llama_index.llms.openai import OpenAI
from llama_index.embeddings.openai import OpenAIEmbedding
from llama_index.vector_stores.chroma import ChromaVectorStore

# ── 1. GLOBAL CONFIGURATION ──────────────────────────────
Settings.llm = OpenAI(model='gpt-4', temperature=0)
Settings.embed_model = OpenAIEmbedding(model='text-embedding-3-small')
Settings.chunk_size = 512
Settings.chunk_overlap = 50

# ── 2. VECTOR STORE SETUP ────────────────────────────────
chroma_client = chromadb.PersistentClient(path='./chroma_db')
collection = chroma_client.get_or_create_collection('my_documents')
vector_store = ChromaVectorStore(chroma_collection=collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)

# ── 3. INDEX — BUILD OR LOAD ─────────────────────────────
if collection.count() == 0:
    print('Building index from documents...')
    documents = SimpleDirectoryReader('./docs').load_data()
    index = VectorStoreIndex.from_documents(
        documents,
        storage_context=storage_context
    )
    print(f'Indexed {len(documents)} documents.')
else:
    print('Loading existing index...')
    index = VectorStoreIndex.from_vector_store(vector_store)

# ── 4. QUERY ENGINE ──────────────────────────────────────
query_engine = index.as_query_engine(
    similarity_top_k=3,
    response_mode='refine'  # most accurate
)

# ── 5. QUERY WITH CITATIONS ──────────────────────────────
def ask(question: str):
    response = query_engine.query(question)
    print(f'Answer: {str(response)}')
    print('\nSources:')
```

```
    for i, node in enumerate(response.source_nodes, 1):
        print(f'  [{i}] Score: {node.score:.3f} | {node.metadata.get("file_name",
"unknown")}')
        print(f'      {node.text[:200]}...')
    return response

# ── 6. RUN ──────────────────────────────────────────────────
ask('What are the main topics covered in these documents?')
```

Section 7 — Question & Answer Reference

Q What is LlamaIndex and what problem does it solve?

LlamaIndex is an open-source Python framework for connecting LLMs to private or custom data. It solves the problem that LLMs only know public training data — they know nothing about your company's documents, internal knowledge bases, or private data.

LlamaIndex provides the full pipeline: loading data from any source, splitting it into chunks (Nodes), embedding them into an Index, and querying with a Query Engine that retrieves relevant chunks and generates grounded answers.

Q What is the difference between LlamaIndex and LangChain?

LangChain is a general-purpose LLM framework built around the concept of chains. It is excellent for building agents, multi-step workflows, and integrating with external tools and APIs. It gives fine-grained control over every component but requires more code.

LlamaIndex is a specialist framework laser-focused on connecting LLMs to data. It thinks in terms of indexes rather than chains. It provides multiple index types, built-in citation tracking, and global configuration. It requires significantly less code for RAG use cases.

They are complementary — many production systems use LlamaIndex as the RAG engine inside a LangChain agent.

Q What is the difference between a Document, a Node, and an Index in LlamaIndex?

Document: the raw data loaded from a source — a PDF file, webpage, or database record. The unprocessed input.

Node: a chunk of a Document after splitting. LlamaIndex splits documents into smaller pieces because LLMs have limited context windows and smaller focused chunks improve retrieval quality. Each Node carries a reference back to its source Document for citation.

Index: a structured representation of all Nodes that makes them efficiently queryable. A VectorStoreIndex stores embedded Nodes in a vector store for semantic similarity search.

Q What are the different index types in LlamaIndex and when do you use each?

VectorStoreIndex: embeds nodes as vectors, retrieves by semantic similarity. Use for most RAG applications where questions require meaning-based understanding.

SummaryIndex: stores nodes sequentially, reads all of them when answering. Use for full document summarisation where you need to process the entire document.

KeywordTableIndex: extracts keywords, matches by keyword overlap. Use for technical or legal documents where exact terminology matters more than semantic meaning.

KnowledgeGraphIndex: extracts entities and relationships, stores as a graph. Use for complex interconnected data where relationships between concepts are important.

Q What is LlamaIndex's Settings object and why is it better than LangChain's approach?

Settings is LlamaIndex's global configuration object. You set the LLM, embedding model, chunk size, and chunk overlap once at the top of your application. Every component — indexes, query engines, retrievers — automatically uses these settings.

In LangChain you must explicitly pass the LLM and embedding model to each component separately. In LlamaIndex, changing the LLM from GPT-4 to GPT-4o is one line — Settings.llm = OpenAI(model='gpt-4o') — and the entire pipeline updates automatically.

```
Settings.llm = OpenAI(model='gpt-4')
Settings.embed_model = OpenAIEmbedding()
Settings.chunk_size = 512

# All components now use GPT-4 automatically — no passing required
```

Q What are Response Modes in LlamaIndex and when would you use refine vs compact?

Response modes control how LlamaIndex synthesises an answer from the retrieved nodes.

compact: the default. Fits as many nodes as possible into a single LLM call. Fast and cost-efficient. Use for most production applications.

refine: generates an initial answer from the first node, then passes each subsequent node to the LLM to refine and improve the answer iteratively. Slower and more expensive but produces the most accurate, detailed answers. Use when accuracy is critical.

tree_summarize: builds a hierarchical tree of summaries bottom-up. Best for very long documents or when you need comprehensive summaries.

simple_summarize: truncates all nodes to fit one LLM call. Use for quick rough answers where speed matters more than completeness.

Q How does source citation work in LlamaIndex?

Every response object from a LlamaIndex query engine includes a source_nodes attribute — a list of the exact NodeWithScore objects that were retrieved and used to generate the answer. Each item contains the chunk text, a relevance score, and the metadata from the source document (file name, page number, etc.).

This is available out of the box with no additional implementation. In LangChain you would need to build a custom chain or use a specific retrieval chain to get similar functionality.

```
response = query_engine.query('What are the risks?')

for node in response.source_nodes:
    print(node.score)                          # relevance: 0.0 to 1.0
    print(node.metadata['file_name'])          # source document
    print(node.text[:300])                     # the chunk used
```

Q How do you persist a LlamaIndex index and why is this important in production?

By default the index is built in memory and lost on every restart. Building an index requires embedding all documents — this is slow and expensive (API cost) for large document collections.

In production you persist the index to disk after the first build and load it on subsequent starts. LlamaIndex supports this with storage_context.persist() and load_index_from_storage(). For scalable production systems you

use an external vector database like ChromaDB, Pinecone, or FAISS as the backend — so the index persists independently and can be shared across multiple app instances.

```
# Build once and save:
index.storage_context.persist(persist_dir='./index_storage')

# Load on restart (fast — no re-embedding):
storage_context = StorageContext.from_defaults(persist_dir='./index_storage')
index = load_index_from_storage(storage_context)
```

Q How would you connect LlamaIndex to an external vector database like ChromaDB or Pinecone?

You create a VectorStore object specific to the database, wrap it in a StorageContext, and pass that storage context when building the VectorStoreIndex. The index then reads and writes to the external database instead of memory.

```
import chromadb
from llama_index.vector_stores.chroma import ChromaVectorStore
from llama_index.core import StorageContext, VectorStoreIndex

client = chromadb.PersistentClient(path='./chroma_db')
collection = client.get_or_create_collection('docs')
vector_store = ChromaVectorStore(chroma_collection=collection)
storage_context = StorageContext.from_defaults(vector_store=vector_store)

index = VectorStoreIndex.from_documents(
    documents, storage_context=storage_context
)
```

Q What is the difference between as_query_engine() and as_retriever()?

as_retriever(): returns a Retriever object that fetches relevant Nodes from the index given a query. It does NOT call the LLM — it only retrieves chunks. Use when you want to inspect what was retrieved, combine multiple retrievers, or build a custom query pipeline.

as_query_engine(): returns a QueryEngine that combines retrieval AND answer generation in one step. It calls the Retriever to get relevant nodes, then passes them to the LLM to generate a final answer. Use for end-to-end question answering.

```
# Retriever only — no LLM call:
retriever = index.as_retriever(similarity_top_k=5)
nodes = retriever.retrieve('What are the risks?')  # returns NodeWithScore list

# Query engine — retrieval + LLM answer generation:
query_engine = index.as_query_engine(similarity_top_k=5)
response = query_engine.query('What are the risks?')  # returns Response
```

Q How would you use LlamaIndex together with LangChain in the same application?

The most common pattern is to use LlamaIndex as a RAG tool inside a LangChain agent. The LangChain agent handles the multi-step reasoning and tool selection; LlamaIndex handles the document retrieval when the agent needs to look something up.

You wrap the LlamaIndex query engine as a LangChain tool using LlamaIndexTool.from_query_engine(). The agent can then call it just like any other tool — web search, calculator, API call — but internally it runs the full LlamaIndex RAG pipeline.

```
from llama_index.core.langchain_helpers.agents import LlamaIndexTool
```

```
# Wrap LlamaIndex query engine as a LangChain tool:
tool = LlamaIndexTool.from_query_engine(
    query_engine,
    name='DocumentSearch',
    description='Search internal company documents for answers'
)

# Use inside a LangChain agent:
agent = initialize_agent(tools=[tool, ...], llm=llm, agent=AgentType.ZERO_SHOT_REACT)
```